

UNDERSTAND THE PROBLEM BEFORE YOU BUILD THE SOLUTION

Creating Software that Matters
with Requirements Engineering



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG

Requirements Engineering

A Brief Introduction (w. additions/comments by Robert Feldt)



Julian **Frattini**, Ph.D.

Chalmers University of Technology, Sweden

<https://julianfrattini.github.io/>



JulianFrattini/**seminar-refun**

Public

Copyright © 2025 Julian Frattini.

This work is licensed under [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/).

Definitions

What exactly are Requirements?

Software Development Lifecycle



Requirements
Specification



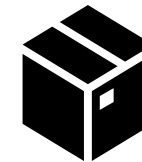
Architecture



Source Code



Test Cases



Product



Requirement & Requirements Artifact

Requirement:

1. A need or constraint imposed by a stakeholder.
2. A capability or property that a system shall have.

(Requirements) Artifact: A documented representation of a (1) need, constraint, (2) capability or property.

REQ1: When a user enters the webpage, the login option shall be highlighted.

REQ2: The system shall be secure and comply to data privacy guidelines

Requirements Engineering

Requirements Engineering (RE) is the systematic, iterative, and disciplined approach to **develop an explicit requirements specification** that all stakeholders agree upon.

Levels of Abstraction

What is the relationship between the following two statements?

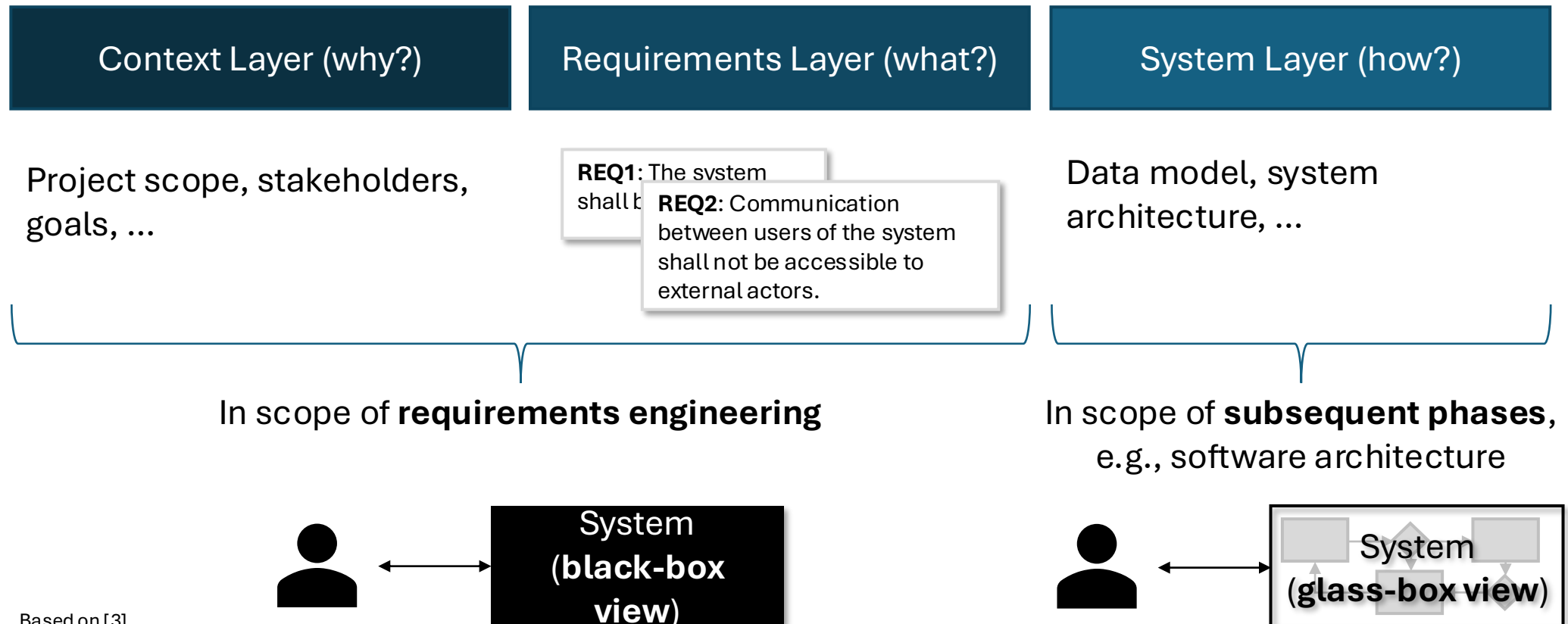
REQ1: The system shall be secure.

is refined to

REQ2: Communication between users of the system shall not be accessible to external actors.

Statements exist on different **levels of abstraction**.

Levels of Abstraction

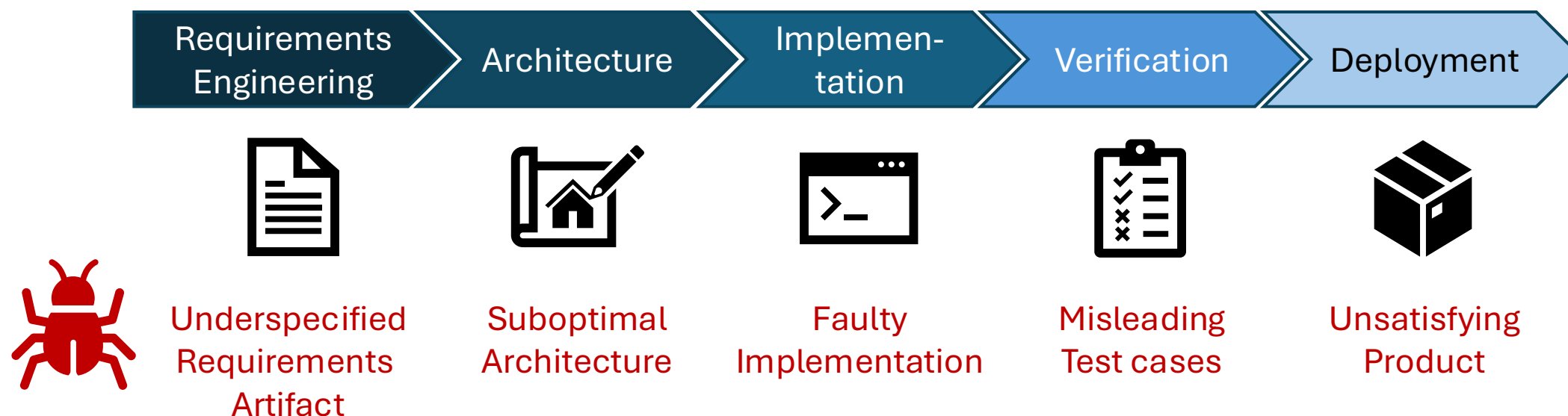


Based on [3]

Impact

Why care about requirements?

Cost of Defect Removal



The cost of removing a defect from an artifact **scales approximately by the factor 10** for each phase that it survives.

Problem- versus Solution-Space



Problem-Space

Why should the system
do something and *what*
should it do?



Solution-Space

How should the system
do it?

Problem- versus Solution-Space

Problem Space

⌞ The system shall be secure.

⌞ The system shall perform well with a large number of concurrent users.

The system's architecture will contain a broker-pattern at the client-server interface with at least 5 subscribed servers

⌞ Large scale maintenance and/or an upgrade shall give the possibility to reach a lifetime of 50 years.

The primary data storage subsystem will adhere to active redundancy.

⌞ All subsystems shall not lose more than 4 hours of acquired or processed measurement data (not yet permanently stored) as a result of an outage in the external power supply.

All communication shall be encrypted with SHA-2.

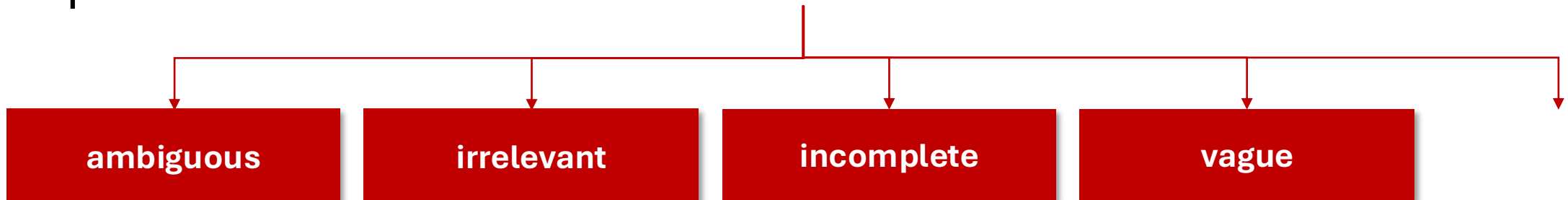
Solution Space

Application

How to Requirements Engineering?

Motivation

It is desirable to specify requirements, but these requirements need to be **free of defects**.



Rather than eliciting requirements all-at-once, we can **incrementally** elicit and refine them.

Techniques



Stakeholder
Elicitation



Goal Modeling



System
Vision



Requirements
Elicitation

Stakeholder Elicitation



A **stakeholder** is a person or a group of persons, interest group, or organization that has to a certain extent **interest in the system** to be developed, or that takes/should take influence on the system's development.



Elementary Steps:

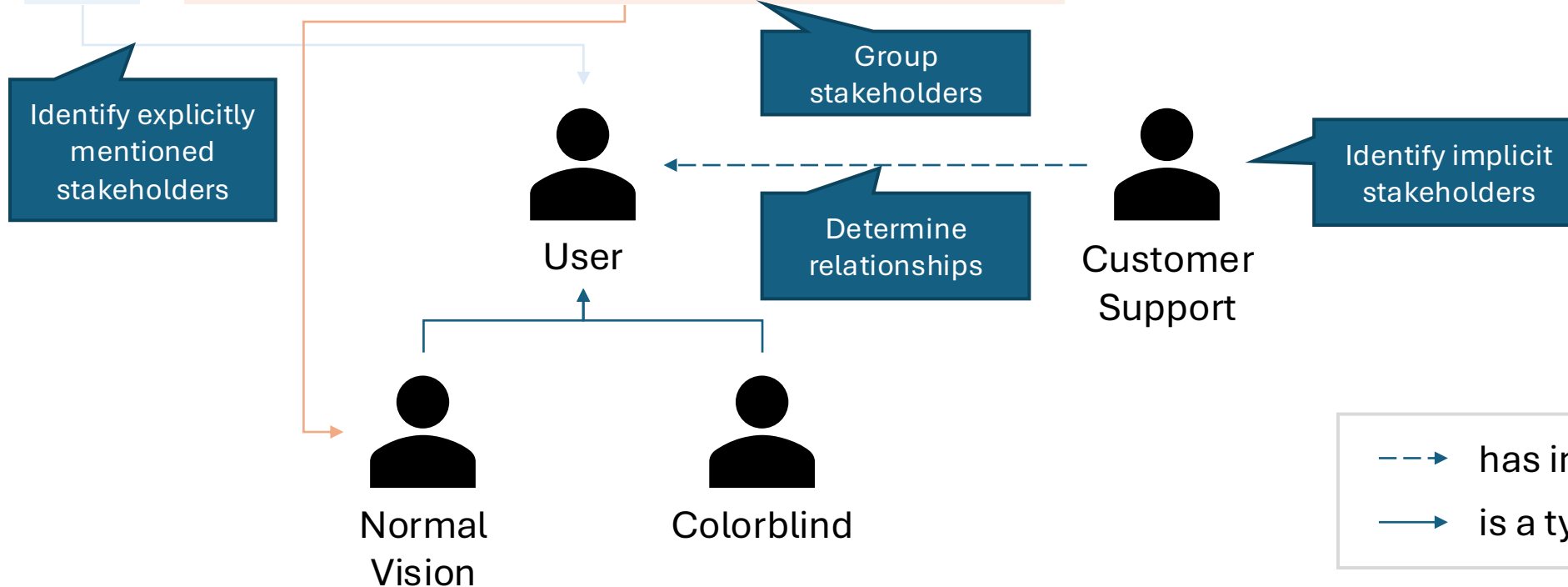


1. **Elicit stakeholders:** list all relevant stakeholders
2. **Elicit relationships:** make relationships between stakeholders explicit

Stakeholder Elicitation



"A user – regardless of their ability to see colors – can log into the system."



Goal Modelling



A **goal** is a prescriptive **statement of intent**, i.e., it describes an abstract property that a system must fulfill.



Goal types:



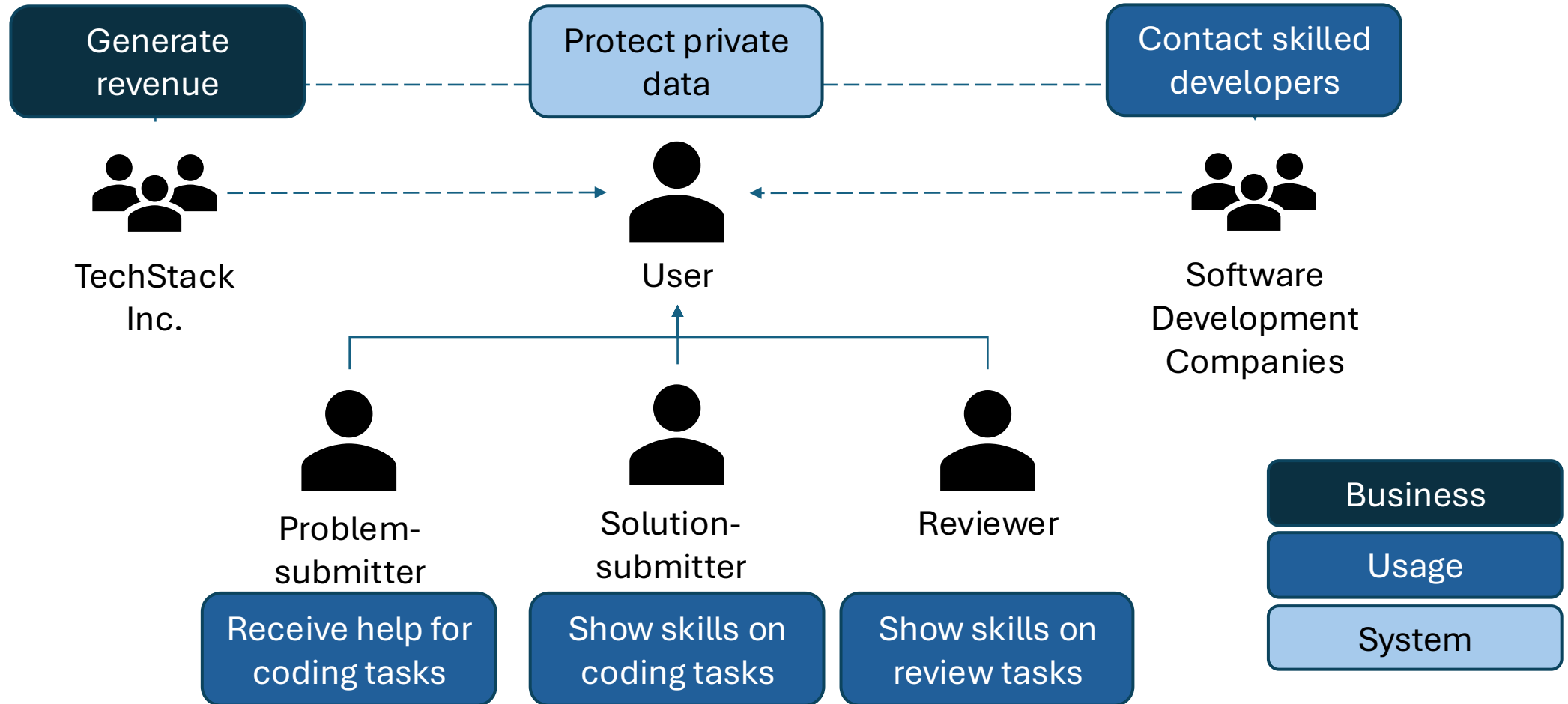
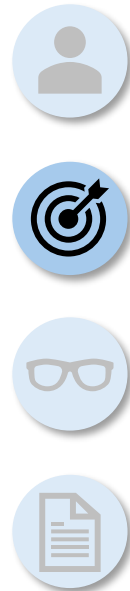
1. **Usage Goals:** Goals with immediate relevance for end users which serve as basis for inference of user requirements



2. **System Goals:** Goals directed at system properties and capabilities (typically quality, i.e., non-functional

3. **Business Goals:** All organization-specific (strategic) goals with relevance to the project

Goal Modelling



System Vision



A **system vision** is the transition point between the context specification and the requirements specification. Its main purpose is to



- give a **comprehensive overview** of the most important *use cases* and



- boundaries, thus it clearly defines the **scope of the system**. It clearly distinguishes which parts belong to the system and which parts are external.



Requirements Elicitation for System Goals



Approach 1: Decompose a system goal into measurable quality requirements along *its aspects*.



The service
should be safe

REQ42: Any request
sent from the server
shall be encrypted.

REQ43: Personal data
shall be removable at
any point in time.

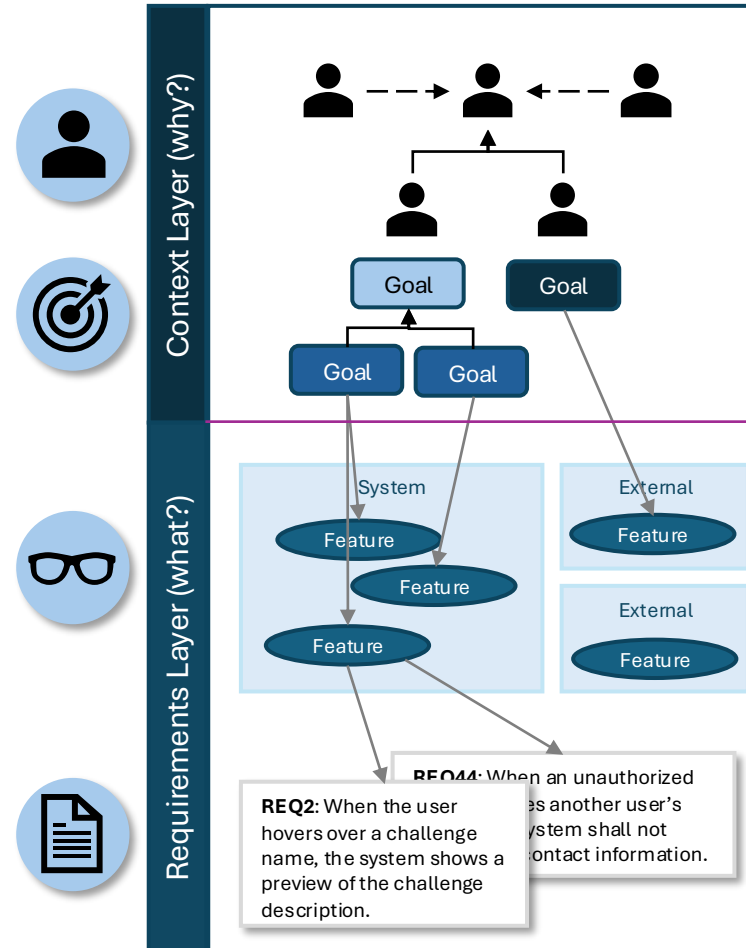


Approach 2: Specify a *misuse-case*, i.e., a functional requirement of what is not supposed to happen.

The service
shall ensure
data privacy

REQ44: When an unauthorized
user accesses another user's
profile, the system shall not
display any contact information.

Requirements Engineering Template





CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG

Requirements Engineering

for **Artificial Intelligence**-based Systems



Julian **Frattini**, Ph.D.

Chalmers University of Technology, Sweden

<https://julianfrattini.github.io/>

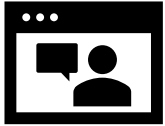


JulianFrattini/**seminar-refun**

Public

Copyright © 2025 Julian Frattini.
This work is licensed under [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/)

Goal



Requirements engineering emerged based on **traditional software**.



Systems including artificial intelligence (AI) **are not traditional software**.

Companies struggle to engineer AI (-enabled) systems.

AI Systems

As an AI system, I will refer to any **data-driven software** (component) throughout this unit

A **data-driven software component** is a piece of software that solves a given task (e.g., image segmentation, sentiment analysis, classification, etc.), using methods from data science, such as machine learning (ML), data mining, natural language processing, signal processing, statistics, etc. The functionality of data-driven software components (or at least part of it) is not entirely defined by the programmer in the classical way (by programming it directly), *but is derived (i.e., learned) from data*.

Siebert, J., Joeckel, L., Heidrich, J., Trendowicz, A., Nakamichi, K., Ohashi, K., ... & Aoyama, M. (2022). Construction of a quality model for machine learning systems. *Software Quality Journal*, 30(2), 307-335.



Traditional Software



requires formal problem



generally transparent



decomposable



AI System



requires no formalization



generally obscure



generally atomic

Requirements Engineering for AI systems (RE4AI) and Machine Learning (RE4ML)

Due to these properties of AI systems, companies struggle to engineer modern systems with traditional SE/RE techniques. Research aims to support companies by working on



**Prioritization of
Software Qualities**



**Development of
Quality Meta-Models**



**Proposal of Data
Quality Attributes**

RE4AI Research:

Prioritization of Software Qualities



AI systems may experience different requirements on their quality. In comparison to traditional software, the following qualities are considered

less important



Usability



Flexibility



Revisability

equally important



Privacy



Reliability

more important



Accuracy



Fairness



Maintainability

Other qualities seem entirely new, like  Re-trainability

Habibullah, K. M., Gay, G., & Horkoff, J. (2023). Non-functional requirements for machine learning: Understanding current use and challenges among practitioners. *Requirements Engineering*, 28(2), 283-316.

Lesson Learned



Systems including artificial intelligence (AI) **are not traditional software** and traditional requirements engineering techniques may not apply to them.



Quality requirements can be **defined, prioritized, and evaluated differently** in AI systems compared to traditional systems.



AI is a solution, so **it does not affect RE directly**. However, when integrating AI into a system, this component will imply specific **requirements on its respective level of abstraction**.



01.06.2026

Two ways RE meets AI

Up to now: AI-as-component => RE4AI

But also: AI-as-developer => RE changes

Agents/AI increasingly doing the **how**

Humans increasingly focus on the **why** and the **what**



Spec-driven development



[Home](#)

[What is Spec Kit](#)

[Quick Start](#)

[Guide](#)

[AI Agents](#)

[Download](#)

[Community](#)

[GitHub](#)

[Get Started](#)

Build High-Quality Software Faster with Spec Kit

Spec Kit revolutionizes software development by making specifications executable. With Spec Kit, transform your ideas into working code using AI-powered specification-driven development.

Formal RE + LLMs: A Two-Way Roadmap

Ferrari & Spoletini (2025) — a vision paper on combining linguistic fluency with formal assurance

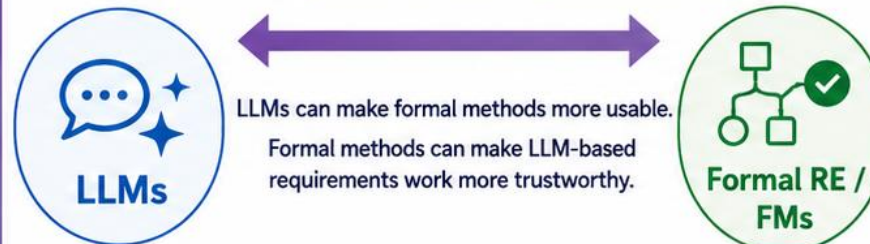
1) How LLMs help formal RE

- 1  Translate between natural language and formal artefacts
- 2  Explain models, formulas, and verification results
- 3  Generate models, code, comments, and tests
- 4  Create trace links across requirements, models, code, and tests
- 5  Extract domain knowledge / ontologies from artefacts

 **Main benefit:** lowers the barrier to using formal methods in normal development.

0) Central Thesis

Complementary weaknesses — productive together



 **Key insight:** when prompts generate code or design models, prompts effectively become requirements.

2) How formal RE helps LLM-based RE

- 1  Constrain and clarify requirements and prompts
- 2  Check ambiguity, inconsistency, incompleteness, and unverifiable claims
- 3  Support correctness through formal specifications and argumentation
- 4  Verify generated artefacts and keep development under control
- 5  Improve trustworthiness, fairness, and assurance

 **Main benefit:** adds control and assurance to LLM-supported requirements work.

3A) What the paper contributes

-  • A bidirectional research roadmap
- A synthesis connecting two communities
- A practical agenda for hybrid RE workflows

3B) What it does NOT claim

-  • Not empirical proof of effectiveness
- Not a validated tool or method
- Not an exhaustive roadmap
- No new dataset or evaluation study

4A) Best fit domains

-  • Safety- or assurance-critical systems
-  • Compliance / traceability-heavy settings
-  • Projects where requirements quality matters

4B) Open challenges

-  • Evaluation without clear ground truth
- Tooling and artefact complexity
- Over-reliance on plausible LLM output
- Scalability and computational cost
- Interdisciplinary adoption barriers

4C) Example hybrid applications

-  • From requirements to verified models
-  • Model- and code-generation with verification gates
-  • Traceable requirements to tests with assurance cases



Takeaway: LLMs can make formal methods usable, while formal methods can make LLM-based requirements engineering trustworthy.